

HOZZ HILLS – PRODUCT REQUIREMENTS AND BACKEND HANDOFF

Document purpose: describe the product and current implementation for the backend engineer taking the system toward production and a split frontend/backend deployment.

Product: Houzz Hills serviced apartments, Kaduna, Nigeria
Status: working local application and payment integration code; not yet connected to production hosting, a live payment merchant account, or a production database

Product timezone/currency: Africa/Lagos / NGN (integer kobo in storage).

1. PRODUCT GOAL

Houzz Hills needs one operational system for public apartment bookings and staff work. A guest should be able to check availability, reserve a stay, pay online, and receive a confirmed booking without a staff member processing the payment. Staff must still be able to create bookings and record payments using their assigned permissions. Owners and managers need a payment register covering accommodation and restaurant transactions, including transfers awaiting confirmation.

The database is the source of truth. Payment confirmation, room availability, stock changes, attendance, and receipts must be written by the server and reflected in the staff workspace after commit.

2. TARGET USERS AND ROLES

The current API has these roles in `src/lib/server/auth.ts`:

- Owner
Current intended access: All property capabilities, including payment confirmation and staff administration
- Manager
Current intended access: Property operations, staff, attendance, restaurant, inventory, and payment confirmation
- Front desk
Current intended access: Reservations and room operations
- Housekeeping
Current intended access: Room operations
- Restaurant cashier
Current intended access: Restaurant POS
- Restaurant manager
Current intended access: POS, menu, inventory, staff, attendance, reports
- Storekeeper
Current intended access: Inventory
- Finance
Current intended access: Read dashboard, POS, payment register, and reports

; cannot
confirm transfers in the current permission table

- Auditor
Current intended access: Read-only operational views listed in the permission table

The backend must enforce permissions on every protected request. Frontend navigation and hidden buttons are convenience only. Accounts are individual, sessions are revocable, and temporary staff passwords require a change before other operations.

3. CURRENT IMPLEMENTATION INVENTORY

=====

“Implemented” below means code exists in this repository. It does not mean the workflow has been exercised with a live provider or deployed in production.

- Public apartment website
Current state: Implemented in the current Next.js app
Relevant implementation: src/app/*

- Public availability
Current state: Implemented against active room inventory and overlapping reservations
Relevant implementation: GET /api/public/availability

- Public booking + online checkout
Current state: Implemented for full-stay payment. Creates a 20-minute hold, initializes hosted checkout, and returns a provider URL. Supports Paystack and Flutterwave through server configuration. Requires a valid guest email.
Relevant implementation: POST /api/public/reservations, src/lib/server/payment-provider.ts

- Online payment confirmation
Current state: Signed provider webhook, provider-side transaction verification, NGN/amount/reference comparison, idempotency, and reservation confirmation are implemented. The return page reads server status and does not confirm payment by itself.
Relevant implementation: POST /api/webhooks/payments, GET /api/public/payments/[reference], /payment-result

- Expired checkout holds
Current state: Secured cleanup endpoint is implemented; the hosting platform still needs a schedule that calls it.
Relevant implementation: POST /api/cron/expire-payment-holds

- Staff authentication
Current state: Implemented with password hashing, database sessions, role permissions, and password-change requirement for temporary accounts. Current implementation depends on Next.js cookies and APIs.
Relevant implementation: src/lib/server/auth.ts, /api/auth/*

- Owner/manager payment register
Current state: Implemented for accommodation payments and restaurant POS orders.
Owner/manager can confirm pending bank transfers; confirmation is audited

- Relevant implementation: `/api/management/payments*`, Payments section in `src/app/management/page.tsx`
- Manual accommodation payment
 - Current state: API records cash, POS terminal, and bank transfer. Transfers remain pending. API rejects manually marking online payments settled. The current UI still shows an “Online” option in the manual payment form; remove or replace this option during frontend cleanup.
 - Relevant implementation: `/api/management/reservations/[id]/payments`
- Staff-created reservations
 - Current state: Implemented. Staff can create reservation records from the workspace. They are separate from public checkout holds.
 - Relevant implementation: `/api/management/reservations`
- Rooms
 - Current state: Basic room creation/list/status operations are implemented.
 - Relevant implementation: `/api/management/rooms*`
- Staff onboarding
 - Current state: User and staff-profile creation with role, employee number, temporary password, and first-login password change are implemented.
 - Relevant implementation: `/api/management/staff*`
- Attendance
 - Current state: Staff can clock in/out; events are stored and duplicate open clock-ins are prevented.
 - Relevant implementation: `/api/management/attendance*`
- Restaurant POS
 - Current state: Menu-backed sales, receipt number, cashier shifts, and cash/POS/bank-transfer modes are implemented. Bank transfer is pending until owner/manager confirmation; a receipt is only returned for settled payments. Recipe-linked stock is decremented on order creation.
 - Relevant implementation: `/api/management/pos*`, `/api/management/menu`
- Inventory
 - Current state: Basic items and stock movements exist, with stock deduction from menu recipes and low-stock visibility.
 - Relevant implementation: `/api/management/inventory`
- Dashboard and updates
 - Current state: Dashboard and event/outbox-backed staff refresh behavior exists. Treat “real time” as committed-data updates/refreshes in the current app, not a proven distributed event platform.
 - Relevant implementation: `/api/management/dashboard`, `/api/management/events`
- PostgreSQL
 - Current state: Schema and ordered migrations exist; local migration `004_online_payments.sql` has been applied.
 - Relevant implementation: `db/migrations/*`, `scripts/migrate.mjs`
- Audit/outbox
 - Current state: Important writes use database transactions and write audit/outbox events.
 - There is not yet a separate notification delivery service.

Relevant implementation: audit_events, outbox_events

PAYMENT POLICY AND BEHAVIOR

- Public online booking charges the full stay total in NGN. There is no deposit/partial-payment selection in public checkout yet.
- The reservation is confirmed automatically only after provider signature validation and server-to-server transaction verification.
- Staff may record cash or terminal payment as settled. A bank transfer remains pending until an owner or manager verifies the company account and confirms it.
- Houzz Hills has specified no refunds. Do not add refund actions or claim them at the system can issue a refund. A cancelled or expired reservation does not reverse a settled payment.
- If a provider reports a successful payment after the room hold expires, the payment is recorded and an exception event is emitted; the reservation is not silently confirmed against possibly resold inventory. Owner/manager exception handling still needs an explicit operational process.
- Restaurant POS currently has manual cash, terminal, and transfer workflows; public hosted checkout is for accommodation bookings. Online POS checkout is not implemented.

4. CORE USER JOURNEYS

=====

4.1 GUEST ONLINE BOOKING

1. Guest enters contact details, stay dates, guests, and room type on /reserve.
2. The server validates dates, checks inventory, locks one available physical room, calculates the full price, stores the guest/reservation, and creates an expiring pending-payment hold.
3. The server starts hosted Paystack or Flutterwave checkout using server-only credentials and redirects the guest to the provider.
4. The provider sends a signed event to /api/webhooks/payments.
5. The API validates the signature, verifies the transaction directly with the provider, checks reference, exact amount and NGN currency, and processes the event idempotently.
6. A valid payment marks the payment settled and the reservation paid/confirmed. The guest return page polls the local payment status; it is informational only.
7. A scheduled call to /api/cron/expire-payment-holds releases holds that expired without payment.

4.2 STAFF-CREATED BOOKING AND MANUAL COLLECTION

1. Authorized staff creates a reservation from the management workspace.
2. Staff records cash or POS terminal as settled, or enters a bank-transfer reference/sender as pending.
3. Owner or manager verifies that the transfer reached the company account and confirms it in Payments.

4. The payment remains visible with actor, method, reference, status, and confirmation audit details.

4.3 RESTAURANT SALE

1. Cashier opens a shift and creates an order from active menu items.
2. The server calculates prices, creates a receipt/order, and deducts recipe inventory atomically.
3. Cash and POS terminal sales are settled; transfer sales are pending confirmation.
4. A paid receipt is available after settlement. Owner/manager confirms transfer through the shared Payments register.

5. FUNCTIONAL REQUIREMENTS AND ACCEPTANCE CRITERIA

BOOKING AND AVAILABILITY

- Public availability must be read from the backend/database, not browser-held inventory.
- Concurrent bookings must not assign the same room to overlapping active stays.
- Public bookings hold a room for a configured short period and release it automatically on expiry.
- Rates, room assignment, guest data, and the payment amount are computed/validated server-side.
- A repeat request or network retry must not create duplicate reservations or duplicate provider charges. A booking-initiation idempotency contract is still needed.
- Staff can create bookings without using public online checkout.

PAYMENTS

- Support one configured online provider per environment, initially Paystack or Flutterwave.
- Provider credentials never reach browser code or frontend environment variables.
- The backend verifies provider signature and transaction state, amount, currency, and reference before settlement.
- Duplicate/retried provider webhooks do not duplicate ledger entries or confirmation side effects.
- Owner/manager can see all accommodation and restaurant payment methods/statuses and confirm pending transfers.
- Pending transfers do not count as settled revenue; settled online payments appear without staff confirmation.
- No refund workflow is allowed by the current business policy.
- Reconciliation/reporting must compare Houzz Hills payment records to provider settlements and company bank deposits.

STAFF, POS, INVENTORY, AND DASHBOARD

- Staff onboarding stores the profile and account under the property, applies

- the assigned role, and requires a temporary password change.
- Staff clock-in/out is attributable, timestamped, and prevents invalid repeated transitions.
- POS finalization is idempotent and stock deduction and order creation are transactional.
- Pending transfer orders cannot receive a paid receipt until confirmed.
- Dashboard metrics and payment totals are derived from committed records and filter by property and business date (Africa/Lagos).
- API authorization is tested for each role, including direct calls that bypass the UI.

6. TARGET ARCHITECTURE: TWO REPOSITORIES AND SEPARATELY HOSTED SERVICES

The current repository is a combined Next.js frontend and backend. To meet the requested deployment model, split ownership while keeping PostgreSQL and payment secrets behind the backend boundary.

Repository 1: houzzhills-web	Repository 2: houzzhills-api
Public website + staff workspace	Auth, booking, operations,
Next.js UI only	REST API + webhook + scheduler
No DB credentials	Owns DB credentials and migrations
No provider secret	Owns provider secrets
HTTPS JSON + cookie auth	
----->	
-- PostgreSQL (private network)	
Payment provider -- signed webhook ----->	
Hosting scheduler -- hold-expiry request ----->	

REPOSITORY 1 – HOUZZHILLS-WEB

- Keep the public marketing/booking pages and authenticated management UI.
- Replace relative same-origin /api/... calls with a configured API base URL such as NEXT_PUBLIC_API_BASE_URL (public URL only; never a secret).
- Remove DB access, payment provider code/secrets, webhook handlers, migration runner, and server business rules from this repo.
- Keep presentation, client-side validation for usability, and API response rendering; the backend remains authoritative.
- Use separate local, staging, and production API base URLs.

REPOSITORY 2 – HOUZZHILLS-API

- Move business logic and endpoint behavior out of Next.js route handlers into an independently deployable TypeScript HTTP API. Fastify, NestJS, or another maintainable framework is a backend-team choice; publish an OpenAPI specification regardless of framework.
- Extract authentication/session logic from Next-specific APIs (next/headers, Response handling) into the API service.
- Own PostgreSQL connection, migrations, role checks, booking/availability tr

- ansactions, payments, POS, inventory, attendance, audit/outbox, and server-side validation.
- Own provider checkout initialization, webhooks, provider verification, and hold-expiration job endpoint.
- Add health/readiness routes and structured logs, metrics, error reporting, rate limiting, and backup/restore procedures.
- API service is the only runtime allowed to connect to the private database.

CROSS-ORIGIN AUTHENTICATION AND HOSTING

- Prefer first-party custom subdomains under the same site, for example app.houzzhills.com for the web app and api.houzzhills.com for the API. Hosting vendors are not selected in this PRD.
- Configure credentialed CORS with an explicit allowlist for production and staging web origins; do not use * with credentials.
- If using the current cookie-session model, set a Secure, HttpOnly cookie with appropriate domain and SameSite attributes; frontend requests must include credentials. Add CSRF protection/origin checks for state-changing cookie-authenticated requests. Validate the exact cross-origin behavior in supported browsers.
- Keep production and staging databases and provider keys separate. Use a private database network and TLS for the API-to-database connection.
- Configure the provider webhook URL to the API host, not the frontend host. Configure the hosting scheduler to call the API hold-expiry route with its bearer secret.
- Never expose DATABASE_URL, provider secret keys, webhook secret/hash, or scheduler secret in the web repository or browser bundle.

7. ENDPOINT CATALOG FOR THE FRONTEND/BACKEND SPLIT

The paths in this table are the current local paths in this repository. The frontend currently calls them on the same origin. The split backend must expose these behaviors over HTTPS and the frontend must consume them through one configured API base URL. The backend engineer may standardize paths under /api/v1; if so, update the OpenAPI contract and frontend together. Do not silently change request/response shapes during extraction.

Callers: Web = public or staff Next.js frontend; Provider = Paystack/Flutterwave server callbacks; Scheduler = hosting platform job. Public means no staff login, not unprotected from rate limits/validation. All management endpoints require an authenticated session and the permission shown.

PUBLIC BOOKING, PAYMENT, SETUP, AND AUTHENTICATION

- GET /api/public/availability?checkIn=YYYY-MM-DD&checkOut=YYYY-MM-DD&guests=N
Caller / access: Web, public

Purpose and current contract: Returns roomTypes[] with room type, rate in kobo, capacity, and available count.
Split status: Exists; frontend consumes it.

- POST /api/public/reservations
Caller / access: Web, public
Purpose and current contract: Body: guest name, email, phone, roomType, checkIn, checkOut, guests, optional notes. Returns reservation reference/amount/status plus checkoutUrl;
public UI redirects there. Creates a room hold and hosted checkout.
Split status: Exists; external API contract and idempotency key still need formalizing.
- GET /api/public/payments/{reference}
Caller / access: Web, public with unguessable reservation reference
Purpose and current contract: Returns only reference, reservation/payment status, and amount; used by payment result page polling. Do not return guest PII.
Split status: Exists; preserve privacy and add throttling.
- POST /api/webhooks/payments
Caller / access: Provider, signature required
Purpose and current contract: Paystack or Flutterwave signed event. Verify signature, then verify successful transaction server-to-server; apply idempotently. Return 2xx for already-processed duplicate event.
Split status: Exists; configure provider URL and verify provider sandbox/live callbacks.
- POST /api/cron/expire-payment-holds
Caller / access: Scheduler, bearer CRON_SECRET
Purpose and current contract: Expires checkout holds past expiry and releases room inventory. Returns count expired.
Split status: Exists; create external schedule and monitoring.
- GET /api/setup
Caller / access: Web, public
Purpose and current contract: Returns whether initial setup is required and enabled.
Split status: Exists; retain one-time safety checks.
- POST /api/setup
Caller / access: Initial owner setup only; local dev or x-setup-secret
Purpose and current contract: Body: propertyName, fullName, email, password. Creates first property/owner exactly once.
Split status: Exists; decide whether production owner bootstrap stays in API or is an operator-only process.
- POST /api/auth/login
Caller / access: Web
Purpose and current contract: Body: email, password; establishes staff session. Login attempts are rate limited.
Split status: Exists; rework cookie setting for independent API domain and add CORS/CSRF handling.
- GET /api/auth/session
Caller / access: Web, authenticated cookie
Purpose and current contract: Returns current user/role or null; frontend uses it to

decide workspace view.

Split status: Exists; preserve response contract during migration.

- POST /api/auth/logout

Caller / access: Web, authenticated cookie

Purpose and current contract: Revokes current session and clears cookie.

Split status: Exists; test cross-subdomain cookie deletion.

- POST /api/auth/password

Caller / access: Web, authenticated cookie

Purpose and current contract: Body: currentPassword, newPassword; changes password and

clears temporary-password gate.

Split status: Exists; retain minimum password policy and session policy.

MANAGEMENT WORKSPACE ENDPOINTS

- GET /api/management/dashboard

Permission: dashboard:read

Purpose / frontend use: Owner/manager snapshot: property, KPI metrics, near-term

reservations, recent activity, housekeeping and attendance counts, user, server time.

Split status: Exists; frontend consumes. Validate every field and role-filtered payload.

- GET /api/management/events

Permission: Authenticated session

Purpose / frontend use: Server-sent event stream of property updates; accepts

Last-Event-ID header or ?cursor=.

Split status: Exists; backend must preserve streaming/reconnect and role filtering or

publish an agreed replacement (SSE/WebSocket/polling).

- GET /api/management/reservations

Permission: reservations:read

Purpose / frontend use: List reservations with guest, dates, unit, status, payment status,

total and paid amount.

Split status: Exists; add query filters, pagination and date-range limits before

production scale.

- POST /api/management/reservations

Permission: reservations:write

Purpose / frontend use: Staff-created booking. Body includes guest name/contact, physical

roomId, dates and guests. Creates a confirmed reservation record; staff collects payment

separately.

Split status: Exists; document all validations and prevent overlapping inventory.

- PATCH /api/management/reservations/{id}

Permission: reservations:write

Purpose / frontend use: Updates reservation status (current UI actions include operational

status changes).

Split status: Exists; define allowed state transitions and audit reasons.

- POST /api/management/reservations/{id}/payments

Permission: reservations:write

Purpose / frontend use: Body: amountKobo, method (cash, pos, bank_transfer), optional

paymentReference, idempotencyKey. Records staff payment; bank transfer is

pending.

Split status: Exists; frontend currently still offers an invalid online selection-remove it.

- GET /api/management/payments

Permission: payments:read

Purpose / frontend use: Combined accommodation and restaurant payment register with source, reference, amount, method, status, recorder and confirmer. Owner/manager use it to reconcile all payments.

Split status: Exists; add pagination, filters, export and separate settled/pending totals.

- PATCH /api/management/payments/{id}

Permission: payments:confirm

Purpose / frontend use: Body: source (accommodation or restaurant). Confirm a pending bank transfer after staff verification.

Split status: Exists; add explicit confirmation audit context and safer concurrency/reconciliation handling.

- GET /api/management/rooms

Permission: rooms:read

Purpose / frontend use: Lists rooms, room types, rates, capacity and current room state.

Split status: Exists; frontend consumes.

- POST /api/management/rooms

Permission: rooms:write

Purpose / frontend use: Creates a room with room number, type, rate in kobo and capacity.

Split status: Exists; consider separate room type/rate-plan resources later.

- PATCH /api/management/rooms/{id}

Permission: rooms:write

Purpose / frontend use: Changes room status; housekeeping role is restricted to cleaning readiness states.

Split status: Exists; define room state transition rules and history.

- GET /api/management/staff

Permission: staff:read

Purpose / frontend use: Lists staff profiles, roles, employment state and last attendance event.

Split status: Exists; restrict personal fields by role and paginate.

- POST /api/management/staff

Permission: staff:write

Purpose / frontend use: Creates user/profile. Body includes name, work email, employee number, department, title, role, temporary password, optional contact/start date.

Split status: Exists; improve temporary credential delivery and account invitation lifecycle.

- PATCH /api/management/staff/{id}

Permission: staff:write

Purpose / frontend use: Updates staff employment status; deactivation should also revoke sessions.

Split status: Exists; verify session revocation on deactivation.

- GET /api/management/attendance
Permission: attendance:read
Purpose / frontend use: Team attendance list/latest events.
Split status: Exists.
- POST /api/management/attendance
Permission: Authenticated user with active staff profile
Purpose / frontend use: Body: eventType (clock_in or clock_out); self clock in/out.
Split status: Exists; add schedule, missed-punch correction and manager approval only if agreed for launch.
- GET /api/management/attendance/self
Permission: Authenticated user
Purpose / frontend use: Returns own current clock state.
Split status: Exists; frontend consumes.
- GET /api/management/inventory
Permission: inventory:read
Purpose / frontend use: Lists active inventory and low-stock indicators.
Split status: Exists; frontend consumes.
- POST /api/management/inventory
Permission: inventory:write
Purpose / frontend use: Current endpoint creates an item or records receive, adjust, or wastage movement. Body shape depends on action.
Split status: Exists; split into clearer /inventory/items and /inventory/movements resources in the versioned API.
- GET /api/management/menu
Permission: pos:read
Purpose / frontend use: Lists active menu items/prices.
Split status: Exists; frontend consumes.
- POST /api/management/menu
Permission: menu:write
Purpose / frontend use: Creates menu item and optional inventory recipe.
Split status: Exists; add update/archive operations and versioned menu behavior.
- GET /api/management/pos
Permission: pos:read
Purpose / frontend use: Returns current cashier shift and recent daily orders.
Split status: Exists; frontend consumes.
- POST /api/management/pos
Permission: pos:write
Purpose / frontend use: Body: items[] (menuItemId, quantity), paymentMethod (cash, pos, bank_transfer), optional transfer reference, idempotencyKey. Creates order/receipt record and stock movements.
Split status: Exists; ensure retry safety and represent pending order lifecycle consistently.
- GET /api/management/pos/{id}
Permission: pos:read
Purpose / frontend use: Returns a receipt only after order payment is settled.
Split status: Exists; frontend consumes for view/print.
- GET /api/management/pos/shift

Permission: pos:read

Purpose / frontend use: Returns current user's open cashier shift.

Split status: Exists.

- POST /api/management/pos/shift

Permission: pos:write

Purpose / frontend use: Body action open + openingFloatKobo, or action close + countedCashKobo; records variance on close.

Split status: Exists; reconcile non-cash tender totals and manager sign-off remain future work.

REQUIRED BACKEND INFRASTRUCTURE ENDPOINTS/CONTRACTS TO ADD OR FORMALIZE

- GET /health/live

Consumer: Hosting platform

Requirement / acceptance: Process is running; no credentials or internal details in response.

- GET /health/ready

Consumer: Hosting platform

Requirement / acceptance: API can reach required dependencies; return non-sensitive readiness status.

- GET /openapi.json (or published OpenAPI artifact)

Consumer: Frontend/backend engineers, API tooling

Requirement / acceptance: Versioned source of truth for every endpoint, schemas, auth, role requirements and errors.

- OPTIONS {path} CORS preflight

Consumer: Browser

Requirement / acceptance: Explicitly allow approved frontend origins, methods and headers with credential support; never wildcard credentialed requests.

- Idempotency-Key support on POST /api/public/reservations and all charge/order writes

Consumer: Web/API

Requirement / acceptance: Same key + same payload returns original result; same key + different payload returns conflict. Persist keys/results so restarts do not lose deduplication.

- Version prefix, recommended /api/v1/...

Consumer: Web

Requirement / acceptance: Add only as part of coordinated OpenAPI/frontend migration; preserve a migration window or update both repos in one release.

- Payment exception/reconciliation listing (extend Payments API or add /api/management/payment-exceptions)

Consumer: Owner/manager

Requirement / acceptance: List late success, amount mismatch, overpayment, provider verification failure and unresolved bank transfer, with safe resolution state. Must not confirm a conflicting room automatically or add a refund action.

- Provider settlement reconciliation (e.g. protected job route or backend worker, exact path TBD)

Consumer: Scheduler/provider API
Requirement / acceptance: Compare provider transactions to local payment records and emit actionable exceptions for missing/mismatched events. Do not rely on webhook delivery alone for financial close.

EXTERNAL ENDPOINTS THE BACKEND ITSELF MUST CALL OR RECEIVE

- Selected Paystack or Flutterwave API
Direction: Backend -> provider
Required operation: Initialize hosted checkout and verify transaction server-to-server.
Secrets remain backend-only.
- Provider webhook callback
Direction: Provider -> backend
Required operation: Deliver signed payment event to backend webhook URL; backend acknowledges only after safe idempotent processing.
- Hosting scheduler
Direction: Scheduler -> backend
Required operation: Call hold-expiry endpoint with bearer secret on configured interval;
alert if calls fail or expired holds accumulate.

8. BACKEND ENGINEER WORK REMAINING

=====

P0 – REQUIRED BEFORE CONNECTING THE SEPARATELY HOSTED FRONTEND

1. Create the backend repository and move API route behavior, server libraries, migrations, and scripts into it; preserve the current API contracts where practical.
2. Publish an OpenAPI contract covering auth, session, booking, availability, payments, rooms, staff, attendance, POS, inventory, events, errors, and pagination.
3. Replace Next-specific auth/cookie/database dependencies with backend-owned equivalents.
Design and implement secure cross-origin session + CSRF behavior for the chosen web/API domains.
4. Add an API base URL configuration to the frontend and remove calls that assume same-origin /api.
5. Add request idempotency for public booking/checkout initialization and all money/order finalization paths. Ensure a guest double-click, timeout, or retry cannot start duplicate charges.
6. Review database concurrency constraints for room overlap and simultaneous payment settlement; document transaction boundaries and locking.
7. Fix manual payment UI so it does not offer “Online” as a staff-recorded method; online status must come from provider verification.
8. Review late-success and overpayment exception handling. Define an owner/manager reconciliation queue that cannot accidentally mark a conflicting stay confirmed.
9. Provide dev/staging/prod environment templates with secrets held by the backend host; configure APP_URL/callback URL and exact CORS origins.
10. Deploy API and managed PostgreSQL independently; run reviewed migrations

and configure
database backups, alerts, and restore procedure.

P0 – PAYMENT ACCOUNT AND LAUNCH SETUP (REQUIRES HOUZZ HILLS ACCOUNT/HOSTING OWNER)

1. Choose Paystack or Flutterwave and complete merchant/KYC onboarding.
2. Set the live secret key only in the backend host secret store; set the web hook hash for Flutterwave as applicable.
3. Configure the provider webhook to <https://api.houzzhills.com/api/webhooks/payments> (adjust path/domain to the final API contract).
4. Configure bank settlement details and reconcile a provider test/live transaction with the company account.
5. Configure a scheduler to call the hold-expiry endpoint every few minutes with Authorization: Bearer
6. Confirm no-refund operating policy and define who handles late payment, overpayment, chargeback/dispute, and payment mismatch exceptions. “No refund” does not remove provider dispute/chargeback obligations.

P1 – PROVE RELIABILITY AND COMPLETE OPERATIONAL WORKFLOWS

- Automated API integration tests for role permissions, room contention, booking retry, duplicate/invalid webhook, provider timeout, late payment, exact NGN amount, bank-transfer confirmation, POS retry, inventory rollback, and hold expiry.
- Provider sandbox end-to-end tests and reconciliation report; do not use live credentials in CI.
- Full role/endpoint authorization matrix tests, especially finance, manager, cashier, front desk, and staff-account deactivation.
- Add API request throttling, validation limits, monitoring/alerts for webhook failures and stale holds, correlation IDs, and a replay/reconciliation procedure.
- Decide and build notifications (email/SMS/WhatsApp) for booking receipt, payment outcome, and operational exceptions. None is presently wired.
- Review payment register access and reporting accuracy; make sure pending amounts are not represented as received revenue.
- Review database retention/privacy, audit export, backups, and production restore exercise.

P2 – LATER PRODUCT SCOPE (NOT REPRESENTED AS COMPLETE)

- POS online hosted checkout, deposits/partial payments, folios/incidentals, rate plans/promotions/taxes, guest self-service, supplier/purchase orders, stock receiving, richer attendance schedules/corrections, housekeeping task assignment, file storage, MFA, and message delivery.
- Financial exports, settlement reconciliation automation, and owner reports with reconciled date/source filters.
- Additional properties/tenancy model if Houzz Hills expands beyond one property.

rty.

9. API, DATA, AND OPERATIONAL CONTRACTS

=====

- PostgreSQL is authoritative. Keep money as integer kobo and currency explicit (NGN). Use Africa/Lagos for property business dates; timestamps should be stored with timezone.
- Existing base schema is in db/migrations/001_initial.sql; no-refund and payment-register changes are in migrations 002 and 003; online-payment indexes are in 004.
- Existing core tables include properties, users, sessions, rooms, guests, reservations, payments, provider_webhook_events, staff_profiles, attendance_events, inventory_items, stock_movements, menu_items, menu_recipes, pos_shifts, pos_orders, pos_order_items, audit_events, and outbox_events.
- Keep API errors predictable (401 unauthenticated, 403 not authorized, 404 missing resource, 409 state/conflict, 422 invalid input, 5xx service failure); avoid returning secrets or unnecessary guest data.
- Paginate list endpoints. Do not depend on in-memory state for sessions, idempotency, event delivery, or booking inventory.
- Define a real-time strategy for separate processes: the current DB outbox/event endpoint can be retained as polling/SSE initially; production should support reconnect cursors and avoid leaking property/role-restricted data.
- Record operational actions in audit and outbox within the same database transaction as the change.

10. RELEASE GATES

=====

Do not call the product production-ready until all are true:

- Frontend and backend deploy independently and communicate through the documented API contract.
- Production DB migrations/backups/restore are operational; DB is not reachable from the frontend host or browser.
- Cross-origin authentication and CSRF protections have been validated.
- Provider sandbox and a controlled live payment reconcile correctly; duplicate webhooks do not duplicate settlement.
- Scheduled hold expiry is active and monitored.
- Automated integration tests pass for money, booking concurrency, permissions, POS/stock, and late/duplicate payment cases.
- Owner/manager can find unresolved transfers and payment exceptions in the register.
- Staff have completed a pilot and there is a documented support/reconciliation procedure.

11. CURRENT VERIFICATION AND KNOWN LIMITS

=====

- npm run lint passed after the current payment-flow work.
- npm run build -- --webpack passed.
- Migration 004_online_payments.sql was applied to the local development database.

- No automated test suite or end-to-end provider test was run as part of the current implementation. No live merchant credentials, production database, hosting service, DNS, or scheduler were configured.
- Existing system design document (docs/houzzhills-system-design.md) describes broader intended product architecture; this handoff PRD is the source for distinguishing currently implemented behavior from future requirements.